

Paidigumal Vivek Patil

G01450948

CS-504 Project

Summary:

The Library Management System project aims to improve the functionality of libraries by automating various tasks related to managing resources, members, and staff. It simplifies processes such as cataloging, borrowing, and returning items, ensuring better organization and accessibility. The system will include features like material cataloging, member registration, borrowing management, author records, staff management, and a querying platform for efficient searching. These functionalities are designed to enhance operational efficiency and provide better service to library users.

Scope:

The project entails developing a database system for public libraries to streamline the management of materials, member information, and borrowing records. It will enable generating reports and analyzing data efficiently. Key components like materials, catalogs, members, authors, and staff will be organized with specific characteristics and connections to ensure smooth library operations. This setup aims to enhance efficiency, support staff in delivering improved service to the community, and maintain an organized resource system.

Material Cataloging:

The system will manage various materials such as books, magazines, e-books, and audiobooks, recording details like title, publication date, genre, and location.

Member Management:

It will maintain records of member names, contact details, and membership initiation dates, providing a straightforward registration process and efficient account management.

Borrowing and Returning Management:

Supports material checkout with monitoring of checkout and due dates, ensuring efficient return processes to update material availability.

Author Records:

Comprehensive data on authors, including names, birth dates, and nationalities, will be recorded, along with mapping relationships between authors and their works.

Staff Management:

Maintains staff records, including job titles, contact details, and hiring dates, while assigning roles to enhance operational efficiency.

Querying Platform:

Provides an intuitive search interface for finding materials by title, author, or genre, with advanced querying options for administrative reports.

ENTITIES AND RELATIONSHIPS:

Entities:

Material:

Represents items available in the library such as books, e-books, etc.

Attributes:

- Material_ID (unique identifier)
- Title
- Publication_Date
- Catalog_ID (a reference to the catalog entry for the material)
- Genre_ID (a reference to the genre of the material)

Catalog:

Records the availability and location of library materials.

Attributes:

- Catalog_ID (unique identifier)
- Name
- Location

Genre:

Categorizes materials into different genres like Fiction, Non-fiction, etc.

Attributes:

- Genre_ID (unique identifier)
- Name
- Description

Borrow:

Details the borrowing activity of members, including dates and staff involvement.

Attributes:

- Borrow_ID (unique identifier)
- Material_ID (reference to the borrowed material)
- Member_ID (a reference to the member who borrowed the material)
- Staff_ID (a reference to the staff who processed the transaction)
- Borrow_Date
- Due_Date

- Return_Date

Author:

Contains information about the authors of the materials.

Attributes:

- Author_ID (unique identifier)
- Name
- Birth_Date
- Nationality

Authorship:

Associates authors with the materials they have created.

Attributes:

- Authorship_ID (unique identifier)
- Author_ID (reference to the author)
- Material_ID (a reference to the material authored)

Member:

Represents the library members who borrow materials.

Attributes:

- Member_ID (unique identifier)
- Name
- Contact_Info
- Join_Date

Staff:

Represents the employees of the library responsible for managing various tasks.

Attributes:

- Staff_ID (unique identifier)
- Name
- Contact_Info
- Job_Title
- Hire_Date

Relationship constraints:

1. Catalog and Material:

- A one-to-many relationship exists between 'catalog' and 'material', where each catalog can contain many materials, but each material belongs to only one catalog. This is indicated by the 'Catalog_ID' foreign key in the 'material' table.

2. Genre and Material:

- Similar to `catalog`, there is a one-to-many relationship between `genre` and `material`, where each genre can be associated with many materials, but each material is categorized under one genre, identified by the `Genre\_ID` foreign key in the `material` table.

3. Authorship, Author, and Material:

- There is a many-to-many relationship between `author` and `material` that is resolved by the `authorship` table. This indicates that authors can author multiple materials and each material can have multiple authors. `Author\_ID` and `Material\_ID` are foreign keys in the `authorship` table, linking to `author` and `material`, respectively.

4. Borrow, Material, Member, and Staff:

- The `borrow` table represents a many-to-many relationship between `member` and `material` and also between `member` and `staff`. A `member` can borrow many materials, a `material` can be borrowed by many members, a `staff` member can process the borrowing of many materials, and a `material` can be processed by many staff members. This is indicated by the `Material\_ID`, `Member\_ID`, and `Staff\_ID` foreign keys in the `borrow` table.

5. Member:

- The `member` table does not directly relate to other tables in this ERD aside from `borrow`, but it represents the entities that can borrow materials from the system.

6. Staff:

- Similar to `member`, the `staff` table is related to other tables through the `borrow` table, representing the staff members who facilitate the borrowing of materials.

Participation constraints:

1. Catalog:

- Partial Participation with Material: Not all catalogs must-have materials associated with them; there may be empty catalogs.

2. Material:

- Total Participation with Catalog: Every material must be associated with one catalog.
- Total Participation with Genre: Every material must have a genre.
- Total or Partial Participation with Authorship: Depending on whether the system allows for materials without authors, this could be total (if every material must have an author) or partial (if there can be materials without a designated author).

3. Genre:

- Partial Participation with Material: There can be genres that currently do not have any materials associated with them.

4. Authorship:

- Total Participation with Author and Material: An authorship record exists to link authors with materials, so it cannot exist without both an author and a material.

5. Author:

- Total or Partial Participation with Authorship: If the system allows authors without any written materials, this would be partial participation. If every author must have at least one material, then it's total participation.

6. Borrow:

- Total Participation with Material, Member, and Staff: A borrow record cannot exist without an associated material, member, and staff member. Therefore, in the context of the borrowing record, the participation of these entities is total.

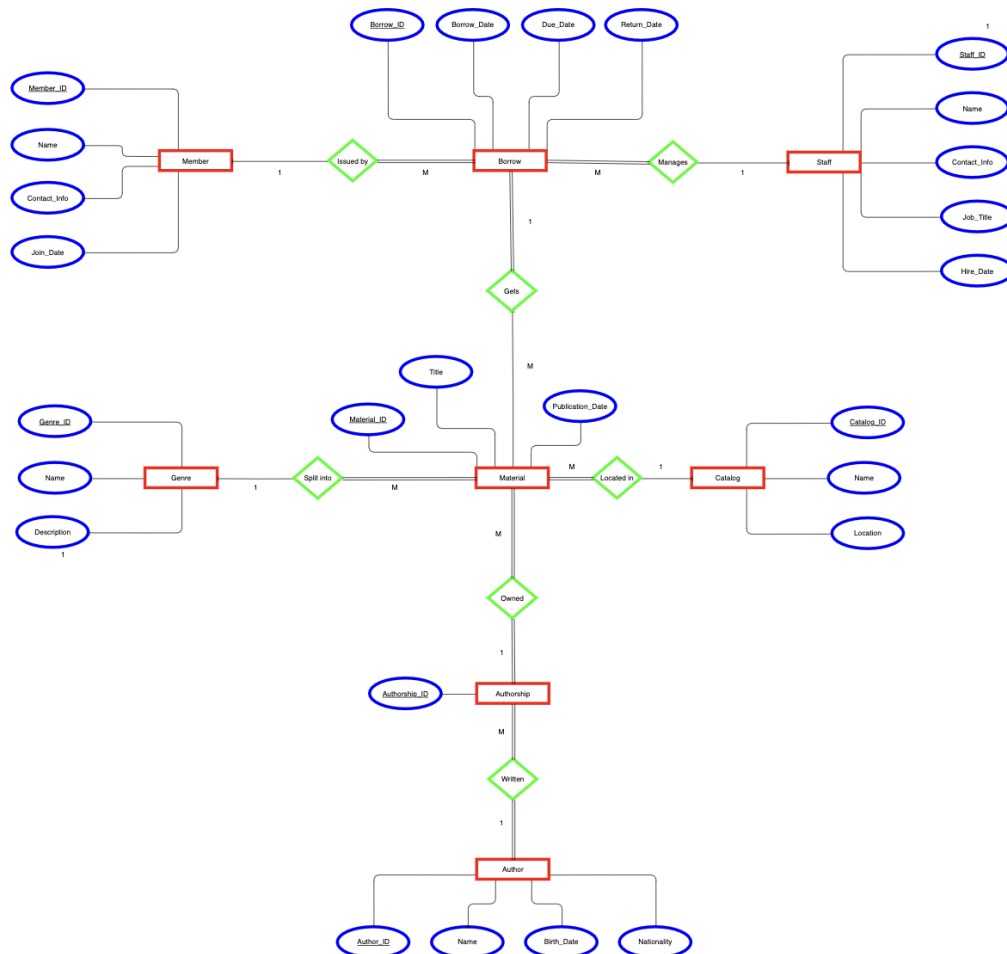
7. Member:

- Partial Participation with Borrow: While every borrower requires a member, not all members are required to have borrow records. Some members may not have borrowed anything.

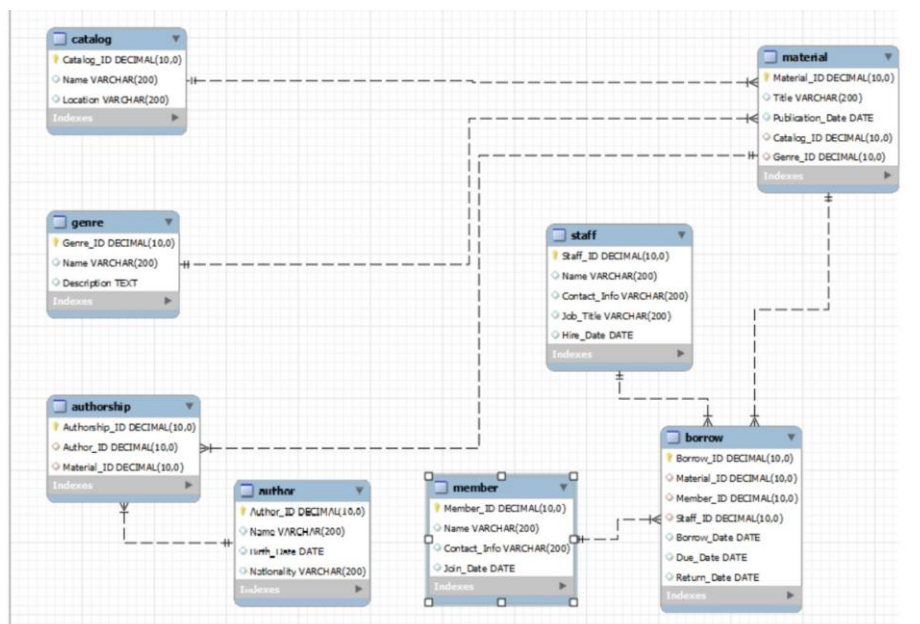
8. Staff:

- Partial Participation with Borrow: Similar to members, not all staff must be associated with a borrowed record, so this participation is partial. A staff may exist without facilitating any borrowing.

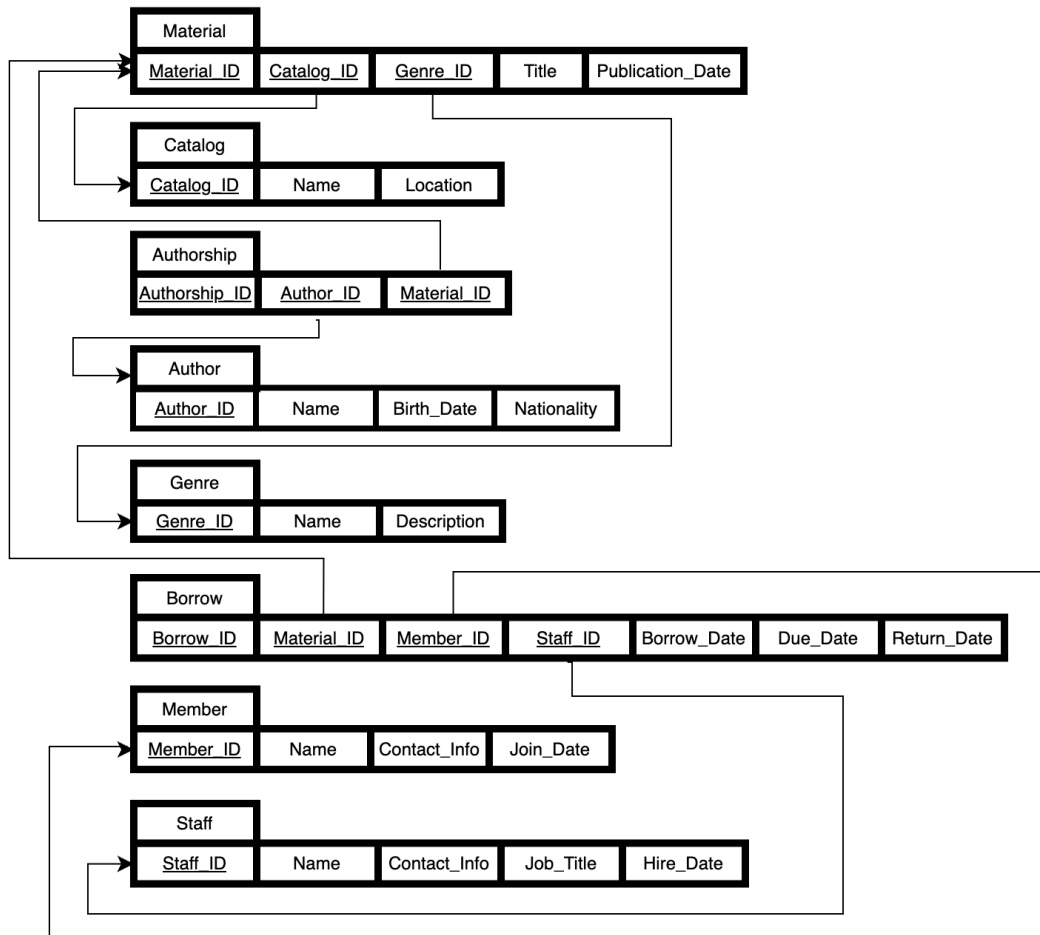
ER Diagram:



System ER Diagram:



Relational Schema:



DATABASE CREATION:

Creating Tables:

Member

```
CREATE TABLE Member (  
  Member_ID NUMERIC NOT NULL,  
  Name VARCHAR(200),  
  Contact_Info VARCHAR(200),  
  Join_Date DATE,  
  PRIMARY KEY (Member_ID)  
);
```

Staff

```
CREATE TABLE Staff (  
  Staff_ID NUMERIC NOT NULL,  
  Name VARCHAR(200),  
  Contact_Info VARCHAR(200),  
  Job_Title VARCHAR(200),  
  Hire_Date DATE,  
  PRIMARY KEY (Staff_ID)  
);
```

Author

```
CREATE TABLE Author (  
  Author_ID NUMERIC NOT NULL,  
  Name VARCHAR(200),  
  Birth_Date DATE,  
  Nationality VARCHAR(200),  
  PRIMARY KEY (Author_ID)  
);
```

Catalog

```
CREATE TABLE Catalog (  
  Catalog_ID NUMERIC,  
  Name VARCHAR(200),  
  Location VARCHAR(200),  
  PRIMARY KEY (Catalog_ID)  
);
```

Genre

```
CREATE TABLE Genre (  
  Genre_ID NUMERIC,  
  Name VARCHAR(200),  
  Description text,  
  PRIMARY KEY (Genre_ID)  
);
```

Material

```
CREATE TABLE Material (  
Material_ID NUMERIC NOT NULL,  
Title VARCHAR(200),  
Publication_Date DATE,  
Catalog_ID NUMERIC,  
Genre_ID NUMERIC,  
PRIMARY KEY (Material_ID),  
FOREIGN KEY (Catalog_ID) REFERENCES Catalog(Catalog_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE,  
FOREIGN KEY (Genre_ID) REFERENCES Genre(Genre_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

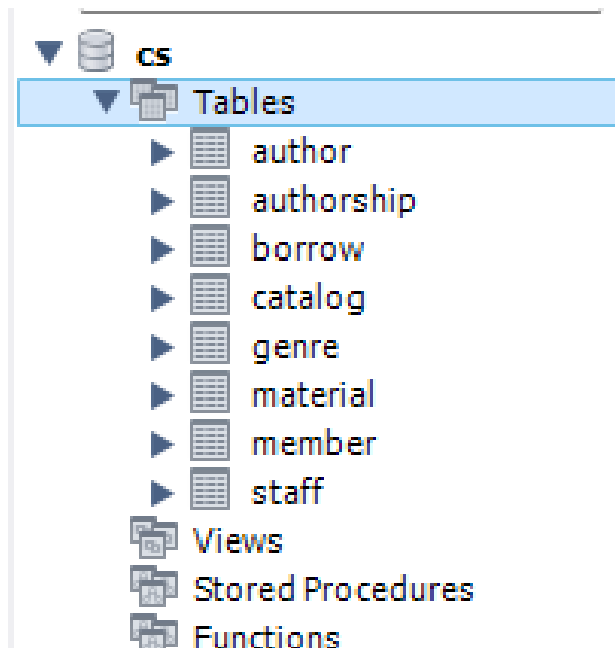
Authorship

```
CREATE TABLE Authorship (  
Authorship_ID NUMERIC NOT NULL,  
Author_ID NUMERIC,  
Material_ID NUMERIC,  
PRIMARY KEY (Authorship_ID),  
FOREIGN KEY (Author_ID) REFERENCES Author(Author_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE,  
FOREIGN KEY (Material_ID) REFERENCES Material(Material_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

Borrow

```
CREATE TABLE Borrow (  
Borrow_ID NUMERIC NOT NULL,  
Material_ID NUMERIC,  
Member_ID NUMERIC,  
Staff_ID NUMERIC,  
Borrow_Date DATE,  
Due_Date DATE,  
Return_Date DATE,  
PRIMARY KEY (Borrow_ID),  
FOREIGN KEY (Material_ID) REFERENCES Material(Material_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE,  
FOREIGN KEY (Member_ID) REFERENCES Member(Member_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE,  
FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)  
ON DELETE CASCADE  
ON UPDATE CASCADE );
```

DISPLAYING THE TABLES OF THE DATA BASE SCHEMA:



Loading Data in Tables:

Author

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Author.csv"
INTO TABLE Author
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES;
select * from Author;
```

Catalog

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Catalog.csv"
INTO TABLE Catalog
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES;
select * from Catalog;
```

Genre

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Genre.csv"
INTO TABLE Genre
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES;
select * from Genre;
```

Material

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Material.csv"
INTO TABLE Material
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
```

```
IGNORE 1 LINES;  
select * from Material;
```

Authorship

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Authorship.csv"  
INTO TABLE Authorship  
FIELDS TERMINATED BY ',' ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 LINES;  
select * from Authorship;
```

Member

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Member.csv"  
INTO TABLE Member  
FIELDS TERMINATED BY ',' ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 LINES;  
select * from Member;
```

Staff

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Staff.csv"  
INTO TABLE Staff  
FIELDS TERMINATED BY ',' ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 LINES;  
select * from Staff;
```

Borrow

```
LOAD DATA INFILE "C:/Users/VIVEK/Desktop/Final Project/Borrow.csv"  
INTO TABLE Borrow  
FIELDS TERMINATED BY ',' ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 LINES;  
update Borrow  
set Return_Date= null  
where Return_Date='0000-00-00';  
select * from Borrow;
```

SQL CODES AND OUTPUTS:

SQL Manipulation (common queries):

1. Search Query

```
161
162 #Search Query
163 • select Name from Member
164 where Join_Date<'2018-12-31';
```

Result Grid | Filter Rows: | Export: | Wra

	Name
▶	Alice Johnson
	Bob Smith
	Carol Brown
	David Williams

2. Insert Query

```
166 #Insert Query
167 • insert into Catalog values (11,'Books1','E11,2');
168 • select * from Catalog where Name='Books1';
169
```

Result Grid | Filter Rows: | Edit: | Export/Import:

	Catalog_ID	Name	Location
▶	11	Books1	E11,2
*	NULL	NULL	NULL

3. Update Query

```
---
170 #Update Query
171 • update Catalog
172 set Name='Magazines1'
173 where Name='Books1';
174 • select * from Catalog where Name='Magazines1';
```

Result Grid | Filter Rows: | Edit: | Ex

	Catalog_ID	Name	Location
▶	11	Magazines1	E11,2
*	NULL	NULL	NULL

4. Delete Query

```
176 #Delete Query
177 • Delete from Catalog
178 where Name='Magazines1';
179 • select * from Catalog where Name='Magazines1';
180
```

Result Grid	Filter Rows:	Edit:	Export/Import:
Catalog_ID	Name	Location	
*	NULL	NULL	NULL

SQL Manipulation (advanced queries):

1. Join Query

```
181 #Inner Join Query
182 • select Material.Title,Catalog.Name,Catalog.Location from Material
183 inner join Catalog on Material.Catalog_ID=Catalog.Catalog_ID
184 where Catalog.Name='Novels';
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Title	Name	Location	
▶ The Picture of Dorian Gray	Novels	H8.1	
The Iliad	Novels	H8.1	
The Old Man and the Sea	Novels	H8.1	

2. Aggregation Query

```
188 #Aggregation Query
189 • select count(Material.Title) as Count,Genre.Name from Material
190 inner join Genre on Material.Genre_ID=Genre.Genre_ID
191 where Genre.Name='General Fiction'
192 group by Genre.Name
193 order by Count desc;
194
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Count	Name		
▶ 14	General Fiction		

3. Subquery Demo

```
197 #Subquery Demo
198 • select Title,Count(Title) from Material
199   where Genre_ID= (select Genre_ID from Genre
200                    where Name='Dystopian & Apocalyptic');
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Title	Count(Title)		
1984	3		

SQL Queries:

1. Which materials are currently available in the library? If a material is borrowed and not returned, it's not considered as available.

```
202 #Q1
203 • SELECT MD.Material_ID,MD.Title
204 FROM Material AS MD
205 WHERE MD.Material_ID NOT IN (
206 SELECT BR.Material_ID FROM Borrow AS BR
207 WHERE BR.Return_Date IS NULL);
```

Result Grid	Filter Rows:	Edit:	Expo
Material_ID	Title		
16	The Adventures of Huckleberry Finn		
17	The Catch-22		
18	The Picture of Dorian Gray		
19	The Call of Cthulhu		
22	A Tale of Two Cities		
23	The Iliad		
24	The Odyssey		
25	The Brothers Karamazov		The Brothers Karamaz
26	The Divine Comedy		
27	The Grapes of Wrath		
28	The Old Man and the Sea		
29	The Count of Monte Cristo		
30	A Midsummer Night's Dream		
31	The Tricky Book		
* NULL	NULL		

```

202 #Q1
203 • SELECT MD.Material_ID,MD.Title
204 FROM Material AS MD
205 WHERE MD.Material_ID NOT IN (
206 SELECT BR.Material_ID FROM Borrow AS BR
207 WHERE BR.Return_Date IS NULL);

```

Result Grid		Filter Rows:	Edit:	Expo
Material_ID	Title			
16	The Adventures of Huckleberry Finn			
17	The Catch-22			
18	The Picture of Dorian Gray			
19	The Call of Cthulhu			
22	A Tale of Two Cities			
23	The Iliad			
24	The Odyssey			
25	The Brothers Karamazov			
26	The Divine Comedy			
27	The Grapes of Wrath			
28	The Old Man and the Sea			
29	The Count of Monte Cristo			
30	A Midsummer Night's Dream			
31	The Tricky Book			
NULL	NULL			

- Which materials are currently overdue? Suppose today is 04/01/2023, and show the borrow date and due date of each material.

```

209 #Q2
210 • SELECT MD.Material_ID,MD.Title,BR.Borrow_Date,BR.Due_Date
211 FROM Material AS MD
212 INNER JOIN Borrow AS BR ON MD.Material_ID = BR.Material_ID
213 WHERE BR.Return_Date IS NULL AND BR.Due_Date < '2023-04-01';

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
Material_ID	Title	Borrow_Date	Due_Date	
20	Harry Potter and the Philosopher's Stone	2021-10-21	2021-11-11	
21	Frankenstein	2021-11-29	2021-12-20	
1	The Catcher in the Rye	2022-12-28	2023-01-18	
2	To Kill a Mockingbird	2023-01-23	2023-02-13	
4	The Hobbit	2023-03-01	2023-03-22	
5	The Shining	2023-03-10	2023-03-31	

3. What are the top 10 most borrowed materials in the library? Show the title of each material and order them based on their available counts.

```
215 #Q3
216 • SELECT MD.Material_ID, MD.Title, COUNT(BR.Material_ID) AS COUNT
217 FROM Material AS MD
218 INNER JOIN Borrow AS BR ON MD.Material_ID = BR.Material_ID
219 GROUP BY MD.Title, MD.Material_ID
220 ORDER BY COUNT DESC LIMIT 10;
221
```

Material_ID	Title	Borrow_Date	Due_Date
20	Harry Potter and the Philosopher's Stone	2021-10-21	2021-11-11
21	Frankenstein	2021-11-29	2021-12-20
1	The Catcher in the Rye	2022-12-28	2023-01-18
2	To Kill a Mockingbird	2023-01-23	2023-02-13
4	The Hobbit	2023-03-01	2023-03-22
5	The Shining	2023-03-10	2023-03-31

4. How many materials has the author Lucas Piki written?

```
223 #Q4
224 • SELECT AD.Name, COUNT(DISTINCT MD.Material_ID) AS COUNT
225 FROM Author AS AD
226 INNER JOIN Authorship AS ASH ON AD.Author_ID = ASH.Author_ID
227 INNER JOIN Material AS MD ON ASH.Material_ID = MD.Material_ID
228 WHERE AD.Name = 'Lucas Piki'
229 GROUP BY AD.Name;
230
```

Name	COUNT
Lucas Piki	1

5. How many materials were written by two or more authors?

```
231 #Q5
232 • SELECT MD.Material_ID, MD.Title, COUNT(AD.Author_ID) AS Number_Author
233 FROM Author AS AD
234 INNER JOIN Authorship AS ASH ON AD.Author_ID = ASH.Author_ID
235 INNER JOIN Material AS MD ON ASH.Material_ID = MD.Material_ID
236 GROUP BY MD.Material_ID, MD.Title
237 HAVING COUNT(AD.Author_ID) >= 2
238 ORDER BY MD.Material_ID ASC;
239
```

Material_ID	Title	Number_Author
22	A Tale of Two Cities	2
28	The Old Man and the Sea	2
29	The Count of Monte Cristo	2
30	A Midsummer Night's Dream	2

6. What are the most popular genres in the library ranked by the total number of borrowed times of each genre?

```
240      #Q6
241      • SELECT GD.Name, COUNT(BR.Borrow_ID) AS Count1
242      FROM Genre AS GD
243      INNER JOIN Material AS MD ON GD.Genre_ID = MD.Genre_ID
244      INNER JOIN Borrow AS BR ON MD.Material_ID = BR.Material_ID
245      GROUP BY GD.Name
246      ORDER BY Count1 DESC;
247
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	Name	Count1
▶	General Fiction	20
	Science Fiction & Fantasy	6
	Horror & Suspense	4
	Classics	3
	Mystery & Thriller	3
	Historical Fiction	1

7. How many materials had been borrowed from 09/2020-10/2020?

```
248      #Q7
249      • SELECT BD.Material_ID, COUNT(DISTINCT BD.Material_ID) AS Count1 FROM Borrow AS BD
250      WHERE BD.Borrow_Date BETWEEN '2020-09-01' AND '2020-10-01'
251      GROUP BY BD.Material_ID
252      ORDER BY Count1 DESC;
253
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	Material_ID	Count1
▶	3	1

8. How do you update the “Harry Potter and the Philosopher's Stone” when it is returned on 04/01/2023?

```
256 #Q8
257 • UPDATE Borrow AS BD
258 SET BD.Return_Date = '2023-04-01'
259 WHERE BD.Material_ID IN (
260 SELECT MD.Material_ID
261 FROM Material AS MD
262 WHERE MD.Title = 'Harry Potter and the Philosopher''s Stone'
263 )
264 AND BD.Return_Date IS NULL;
265 • SELECT * FROM Borrow WHERE Return_Date='2023-04-01';
```

Result Grid

	Borrow_ID	Material_ID	Member_ID	Staff_ID	Borrow_Date	Due_Date	Return_Date
▶	20	20	7	2	2021-10-21	2021-11-11	2023-04-01
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

9. How do you delete the member Emily Miller and all her related records from the database?

```
267 #Q9
268 • DELETE FROM Member WHERE Name='Emily Miller';
269
270 • SELECT * FROM Member WHERE Name='Emily Miller';
271
```

Result Grid

	Member_ID	Name	Contact_Info	Join_Date
*	NULL	NULL	NULL	NULL

10. How do you add the following material to the database?

```
#Q10
BEGIN;
INSERT INTO Material (Material_ID, Title, Publication_Date, Catalog_ID, Genre_ID)
SELECT (SELECT MAX(Material_ID) + 1 FROM Material), 'New Book', '2020-08-01', (SELECT Catalog_ID FROM Catalog WHERE Name = 'E-Books'),
      (SELECT Genre_ID FROM Genre WHERE Name = 'Mystery & Thriller');

INSERT INTO Author (Author_ID, Name)
SELECT (SELECT MAX(Author_ID) + 1 FROM Author), 'Lucas Luke';

INSERT INTO Authorship (Authorship_ID, Author_ID, Material_ID)
SELECT (SELECT MAX(Authorship_ID) + 1 FROM Authorship), (SELECT Author_ID FROM Author WHERE Name = 'Lucas Luke'),
      (SELECT MAX(Material_ID) FROM Material);
COMMIT;
```

```
286 • SELECT * FROM Material
287     WHERE Title = 'New Book';
288
289
```

Result Grid					
Filter Rows:					
Edit:    Export/Import: 					
	Material_ID	Title	Publication_Date	Catalog_ID	Genre_ID
▶	32	New Book	2020-08-01	3	2
*	NULL	NULL	NULL	NULL	NULL

```
290 • SELECT * FROM Author
291     WHERE Name = 'Lucas Luke';
```

Result Grid				
Filter Rows:				
Edit:    Export/Import: 				
	Author_ID	Name	Birth_Date	Nationality
▶	21	Lucas Luke	NULL	NULL
*	NULL	NULL	NULL	NULL

```
297 • SELECT * FROM Authorship
298     WHERE Author_ID=21;
299
300
301
302
```

Result Grid			
Filter Rows:			
Edit:   			
	Authorship_ID	Author_ID	Material_ID
▶	35	21	32
*	NULL	NULL	NULL

EXTENDED FEATURES:

1. Alert staff about overdue materials on a daily-basis?

- By merging the Material and Borrow tables using an Inner Join and including a where condition, you may retrieve the titles of materials whose due date has already gone and their return date is null.

```
SELECT MD.Title
FROM Material AS MD
JOIN Borrow AS BR ON MD.Material_ID = BR.Material_ID
WHERE BR.Due_Date < CURRENT_DATE
AND BR.Return_Date IS NULL;
```

2. Automatically deactivate the membership based on the member's overdue occurrence (>= three times). And reactivate the membership once the member pays the overdue fee.

- This query inserts data into the Membership_Record table by selecting the Member_ID from the Member table and setting NextDueDate to '2024-05-20', MembershipStatus to 'Active', LatePaymentsCount to 0, and IsPaymentRecent to 'Yes'.

```
CREATE TABLE Membership_Record(
MemberID NUMERIC NOT NULL,
NextDueDate DATE,
MembershipStatus VARCHAR(30),
LatePaymentsCount NUMERIC,
IsPaymentRecent VARCHAR(30),
PRIMARY KEY(MemberID),
FOREIGN KEY(MemberID) REFERENCES Member(Member_ID)
ON UPDATE CASCADE
ON DELETE CASCADE
);
```

- This query updates the Membership_Record table, setting MembershipStatus to 'Deactive', incrementing LatePaymentsCount by 1, and setting IsPaymentRecent to 'NO' for records where NextDueDate is before the current date and LatePaymentsCount exceeds 3.

```
INSERT INTO Membership_Record
SELECT (SELECT Member_ID from Member), '2024-05-20', 'Active', 0, 'Yes'
FROM Membership_Record;
```

```
SET MembershipStatus = 'Deactive',
LatePaymentsCount = Overdue_Count + 1,
IsPaymentRecent= 'NO'
WHERE NextDueDate < CURRENT_DATE
AND LatePaymentsCount > 3;
```

- This query updates the Membership_Record table, setting MembershipStatus to 'Active' and resetting LatePaymentsCount to 0 for records where IsPaymentRecent is 'Yes'.

```
UPDATE Membership_Record  
SET MembershipStatus = 'Active',  
    LatePaymentsCount = 0  
WHERE IsPaymentRecent = 'Yes';
```